Study Circles

Student Name: Jamie Hanson
Date Submitted:
Course: Stage 6 Software Engineering
GitHub Link:

0. Table of Contents

---

1. Identifying and Defining

1.1 Problem Statement

Outline the problem or opportunity that the project addresses. Consider who is affected by this issue. **Explain** why this problem or opportunity is significant.

Recently in Australia, the Federal government came out with new age rules regarding social media. This poses an issue to high school students who are under 16, or in a study group / group project with a student or students who are under 16 years old. Previously high school students -- including myself -- use services like instagram or snapchat to create group chats, share study-materials, and organise group study sessions; however now due to the new restrictions these services no longer provide a valid way to communicate with peers who are under 16 -- which can be a big issue in group project tasks. According to Authors; Carlos C. Goller, Micah Vandegrift, Will Cross, and Davida S. Smyth, of the American Society of Microbiology journals, Journal of Microbiology & Biology Education, state that “Sharing Notes is encouraged”^*1, and encourage sharing notes and collaborative effort study as a way to “increase student engagement”

^*1 Goller CC, Vandegrift M, Cross W, Smyth DS.2021.Sharing Notes Is Encouraged: Annotating and Cocreating with Hypothes.is and Google Docs. J Microbiol Biol Educ.22:10.1128/jmbe.v22i1.2135.https://doi.org/10.1128/jmbe.v22i1.2135

Justify the development of a software solution as an appropriate response.

StudyCircles aims to solve this problem of a lack of inclusive study group supplementary applications by providing a program that allows students of all ages to create study groups, share study-materials, organise group study sessions, and plan around related commitments in a standalone messaging application - in a way that is void of the current Australian restrictions regarding social media. Supplementing students' loss of access to platforms where they could already access these materials in a social setting, with a more formal and moderated platform, encouraging students to collaborate effectively -- increasing their student engagement.

1.2 Project purpose and Boundaries

Outline what the project is trying to achieve.

StudyCircles is trying to provide a safe, secure platform for students from years 7 through to the end of their education -- whether that be year 10, year 12, or even university --; with fast uploads and instant communication encrypted and sent through a central server. As well as an easy to use/intuitive interface that allows students to interact with multiple study groups and self-moderate these chats without a third-party ever needing to access the chat.

1.3 Stakeholder Requirements

Identify stakeholders (client, users, teacher, peers).

Describe their needs, expectations, and how these influenced the project direction.

The main stakeholders for this project would be the students/users themselves. Due to Australian social media restrictions students find themselves with the need for a new platform to organize their study groups as they have to move away from instagram, snapchat and other social media platforms for organising study groups with students under 16 years old. They'd require a fast and instant application to keep up with the group chats that they were used to before these restrictions took place, and also a simple yet pleasing interface for using the program. This influenced the project direction greatly; as the main need for the program came due to the new social media restrictions for under 16s, the program needs to ensure that it remains compliant with the restrictions and falls outside of the restricted category -- meaning that it needs to be more focused on instant messaging people *the user knows* rather than sharing their experiences with a collection of users (random or known). This influenced various key parts of the project's direction, from ideas on how to connect users to one another to what the chatting part of the platform should look like.

1.4 Functional Requirements

List and **describe** what the system must do.

For StudyCircles to achieve it must meet the following Functional Requirements:

- Allow users to easily find and add other users -- This needs to be private to follow the Australian Social Media restrictions, Users should have a 6-8 digit "friend code" that is kept private to all non-friends.
- Allow users to easily add their friends to a study group - This needs to be easily done and check whether or not the user is the correct age.
- Allow users to moderate chats -- give users ability to kick and mute users as well as delete user messages that could have unsafe content.
- Allow users to block other users -- Allows for a more secure system that stops users from being harmed by bad actors misusing the project.
- Allow users to upload files -- Needed to share study materials from PDFs, audio, video, and any other format through encrypted channels.
- Allow users to send messages -- Give users the ability to send and receive encrypted messages within the individual study groups -- private messaging should be blocked.
- Allows users to select their age -- Needed for selecting whether a chat required an over 18 moderator or not.

1.5 Non-Functional Requirements

List and **describe** system qualities such as:

- Performance
- Usability
- Security
- Reliability

For StudyCircles to achieve it must meet the following Non-Functional Requirements:

- Quick and (Near)-Instant messaging -- akin to social medias like Instagram and Snapchat.
- A wide range of accepted file formats -- needed for sharing.
- Clean and simple visuals / UI with intuitive layout
- Basic sounds for better user engagement and experience
- Encryption across all forms of messaging
- Highly reliable -- 1% Of all messages should be lost due to any errors whether it be bad internet resulting in packet-loss, or high server-load.

1.6 Constraints

Identify limitations affecting the project, such as:

- Time
- Technical knowledge
- Hardware/software access

This project has quite a few constraints due to it being a school project;
For one this project has a time constraint of the start of this term -- term 2 -- until the end of next term, being approximately 20 weeks, however all of the practical is expected to be completed within the next term so the programming part of this project is limited to 10 weeks.
Additionally only free resources / libraries or APIs will be used, however this shouldn't pose a problem considering the StudyCircles application doesn't need any API's in its current scope.
The project is also quite complex, involving message encryption, RSA Handshakes, and socket communication, meaning I'll have to spend time looking at packet sending and handling between multiple clients in a server-based system.
StudyCircles end product will be limited due to the schools restrictions on opening ports and servers, meaning that I'll have to showcase and ship the project as a localhost only solution -- having a server on localhost and 2 clients open on the same machine, this could be fixed if someone had knowledge of port forwarding on their home computer however.
The only other type of constraint limiting StudyCircles is the legislation that caused the problem that StudyCircles is trying to solve. StudyCircles needs to avoid being seen/labeled as a social media by the Australian Government, avoiding public interfacing in large channels / threads, and anything social-media like.

1.7 Requirements Analysis and Prioritisation

Analyse the functional and non-functional requirements. In your analysis, consider:

- which requirements were prioritised and why,
- trade-offs made due to constraints,
- how requirements align with the identified problem or opportunity

There are various Non-Functional and Functional requirements chosen and some of these are of higher priority than others; however there aren't any major trade-offs made due to the constraints of StudyCircles as StudyCircles goal was to be built around them. The main priorities of StudyCircles is to be fast/well-optimised, intuitive in all its systems -- from adding other users, moderating chats, blocking users, uploading files and managing study groups. The main focus of StudyCircles systems will be the careful implementation of adding and blocking users, as StudyCircles is meant to supplement students learning in the aftermath of the Australian Government's social media restrictions -- and as such StudyCircles needs to avoid being labelled as a social media to remain effective in its solution.

---

2. Research and Planning

2.1 Development Methodology

Describe the development approach used (e.g. Agile, Waterfall, WAgile).
Justify the suitability of this methodology. You could consider…

- Project size and complexity
- Time constraints
- Feedback and iteration requirements

Study Circles is going to use the Agile approach to Software Development, which is an Agilite, Adaptive, Progressive, and Collaborative methodology focusing on breaking down software into sprints. Each step of the development cycle is labelled as a sprint and has its own set goal, whether that be defining requirements or design, to the creation of integral parts of the final system. Allowing for a more modular approach to software development.

Agile development best suits Study Circles for various reasons, its focus on speed is a huge help in mending any trade-offs caused due to time-constraints, and allowing for more time to experiment with packet handling etc… Study Circles also doesn't have a huge range of different ideas -- in that most of the ideas are focused on the same area -- meaning that coherence isn't a huge issue between features. Agile also helps with the development of new systems, I haven't had a lot of experience with pythons socket library and I expect my end product to be different depending on what use cases it suits best.

2.2 Tools and Technologies

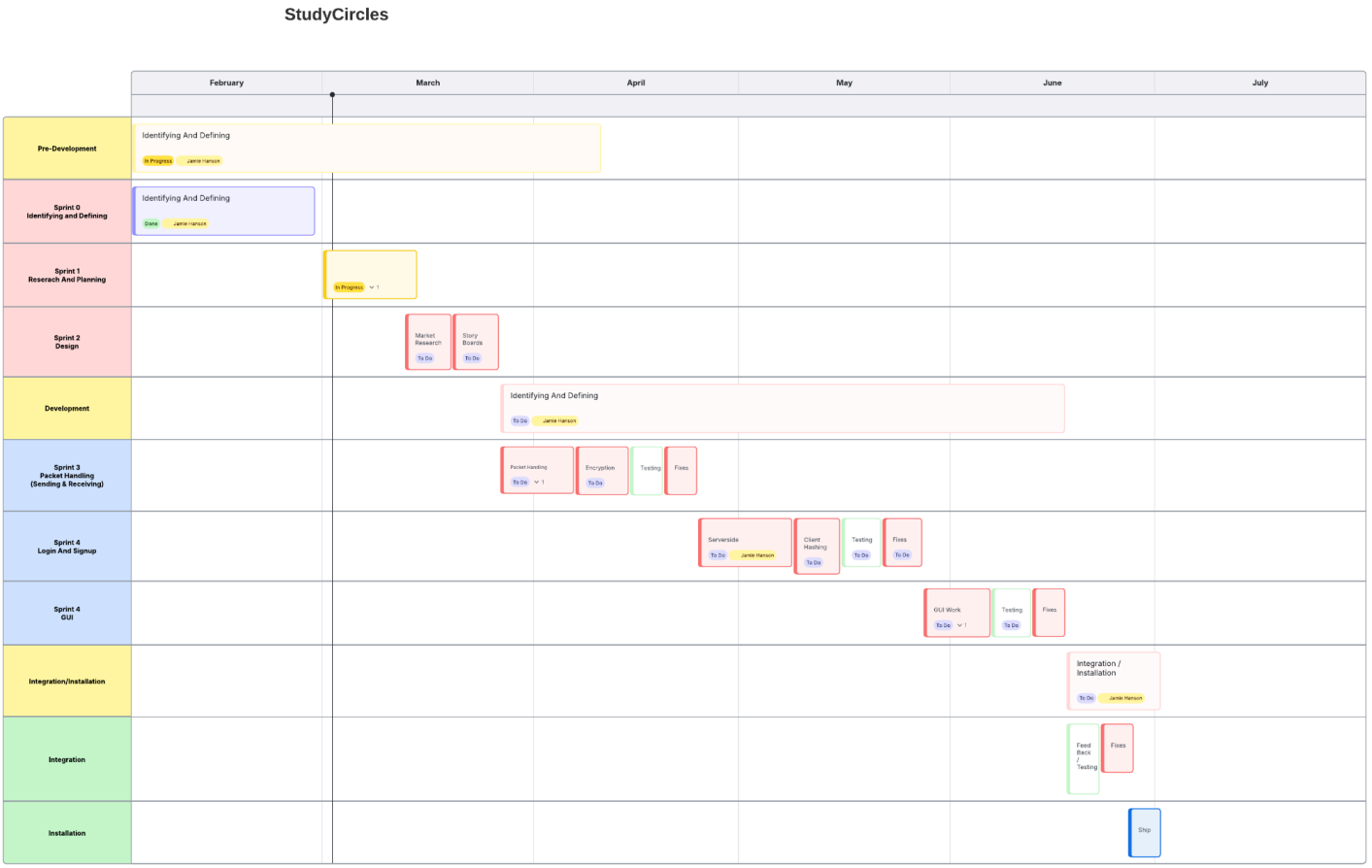
Justify the selection of software applications, engines, developer tools, programming languages, IDEs, frameworks, libraries and/or hardware components.
Explain how these tools supported efficient and effective development.

Throughout the development cycle of Study Circles, I plan to use a select few libraries and tools to better its development. I plan to use Python for Study Circles as it's the language I have the most experience with when it comes to standalone applications. Next is the IDE (Integrated Development Environment), I plan to use PyIdle as it's barebones nature leads to a stable development environment with predictable results, it also ensures no problems rise from unrelated plugins -- which would be an issue if I used PyCharm or VSC for example. There are 3 main libraries I plan to use during StudyCircles' development, namely Sockets -- for requests across client and server, sending messages to and fro -- Cryptography -- for its support for Asynchronous encryption as well as generating hashes and checks useful for storing passwords, and sending over synchronous keys for encrypted communication -- and TKinter -- for it's easy to use OOP style GUI systems, I also have quite a bit of experience with TKinter after my previous projects with Asynchronous image loading and web requests etc. Lastly I plan to use a mix between Goodnotes -- my main note-taking / drawing app on iPad -- Excalidraw, and Figma for design, whether that be for StoryBoard wireframes or Timeline development as I have experience with all 3 of them and they have clean end results.

All these tools will lead to a more efficient and effective development cycle of StudyCircles - from well documented libraries to systems that I have extensive experience in - they all coincide towards a more coherent and cleaner end result / end user experience due to their ease-of-use and high-power features (This goes for all of the tools listed).

2.3 Gantt Chart / Timeline

Include a timeline showing key project milestones.



Explain how time was allocated to planning, development, testing, and evaluation.

In this Gantt Chart time has been split up into 3 sections, Allocated to Pre-Development (Planning), Development, Integration/Installation (Testing / Evaluation / Installation). The majority of the time goes into development which is split into 3 sprints, Packet Handling for implementing server architecture and designing what information can and should be sent through packets as well as Encrypting these packets and establishing Handshakes, Login and Signup for handing user registration and data management on server-side as well as password hashing on the client, The last sprint is for the GUI which is handling all this information and compiling it into an intuitive TKinter based GUI application. I allocated time from the 27th of March to the 17th of June for this stage of the project as it involves working with systems I'm unfamiliar with as well as implementing complex systems with important security requirements. The second most timely part of this project is the Identifying and Defining section, which takes us for the most part up to today, being split into 3 parts itself, Sprint 0 being Identifying and Defining (The details from earlier in this document), Sprint 1 being Researching and Planning (this section of the document) and the third section being Design -- linking to the System Design point of this project -- Having a few days allocated to each point to match my class schedules and indicate the time it's taken so far. Lastly is the Integration/Installation segment of the Gantt Chart which takes us up until the 1st of July from the 17th of June, Involving testing/feedback with peers - this is where I will also evaluate the project) - final fixes, and shipping the project as a final deliverable in a zip format with a readme.md and a requirements.txt alongside the project files.

2.4 Communication Plan

Explain how client or peer feedback was obtained and incorporated.

Client / Peer feedback will be obtained at every “testing” stage of the project -- as defined in the Gantt chart -- where Stakeholders (students / my classmates) will test the project and give a PMI (3 points for Plus, 3 for Minus and 1 for interest) in a simple .txt file, through this will then be incorporated within the allotted time for “fixes” for that category, however any major changes will be avoided. This will lead to a more coherent experience for users, ensuring that the end product is more user friendly and readable.

2.5 Resource Allocation Justification

Justify the resource allocation for the project, including:

- Time
- Software and hardware tools
- Human input (client, peers, teacher feedback)

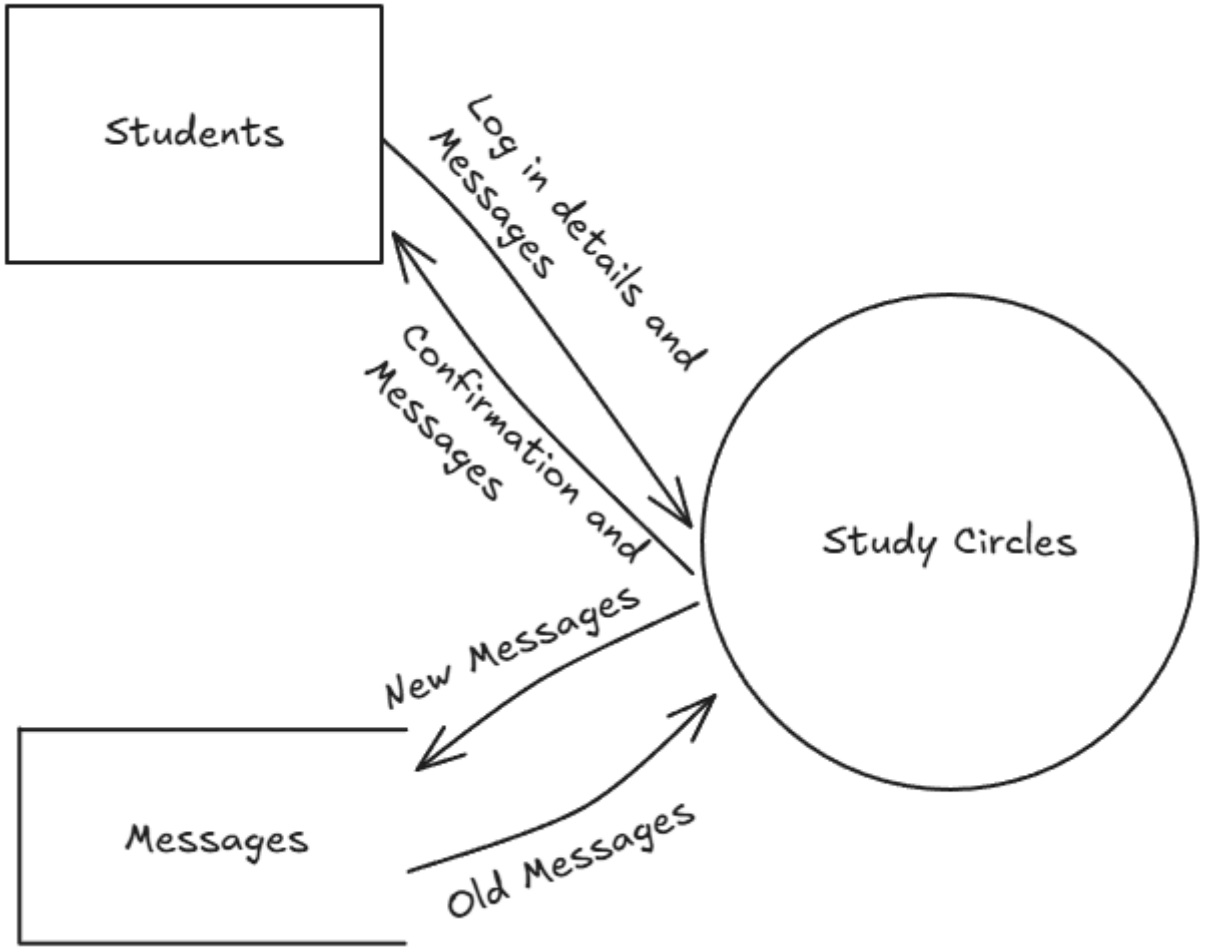
Resources for this project have been allocated due to various key factors around the time for each component of the task / sprint, the tools that will be used as well as when / where in the project human input will be taken. Firstly is time, Time has been allocated mainly to the Development step, this is because at the time of making the Gantt chart all of Identifying and Defining had been completed as well as a large portion of Research and Planning; it’s also because I suspect that Producing and Implementing will take quite a long time -- As I’m working with libraries I haven’t used before (In comparison to making the same types of diagrams I’ve made several times in the past). As for software and hardware tools, I’ve chosen Python, Sockets and TKinter, these have been chosen due to their wide relevance in the programming space -- sockets being the standard for networking with python -- and due to my experience with them -- having worked with Python and TKinter extensively in the past. Lastly is Human Input, human input will be taken at the end of each stage of development, allowing for a day or two to fix the issues that come up in testing or from their feedback, human input will come from the clients (A.k.a my peers) due to how easily I can get them to test it.

---

3. System Design

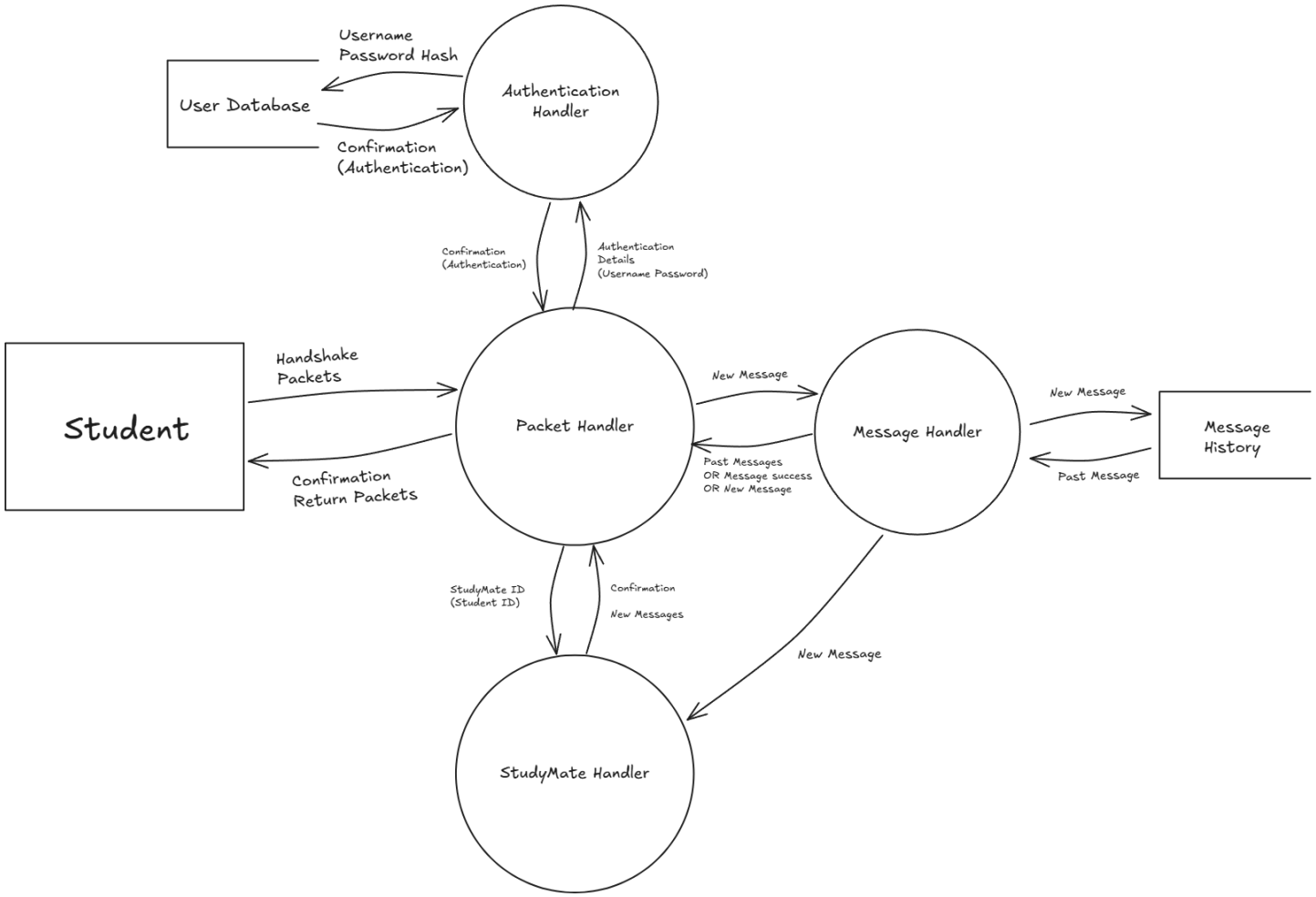
3.1 Context Diagram

Include a context diagram showing system boundaries and external entities.



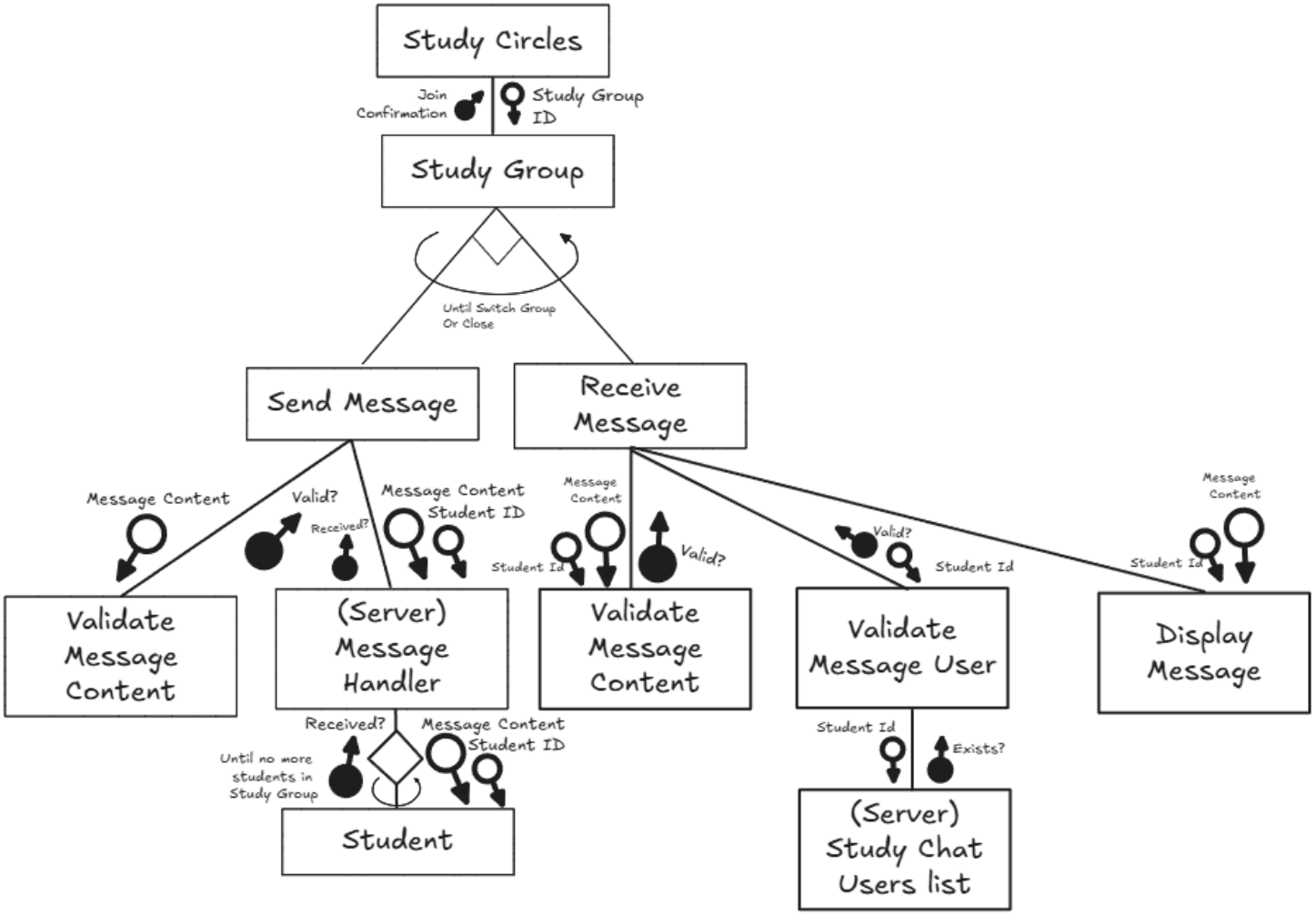
3.2 Data Flow Diagrams (Level 1)

Illustrate how data moves through the system.



3.3 Structure Chart

Show the modular structure of the system and relationships between modules.



3.4 IPO Chart

Input	Process	Output
Username, Password	Hash the password and check both the username and hashed password to the database of users on server, check to see if the information matches	Valid Username and Password => Log the user in. Invalid Username OR Password => Display error message
Student ID	Check to make sure student ID is NOT the users, then find that user	Study Request sent to that student
StudyGroup Code	Check to make sure that study group exists.	Join request sent to that study group
Message, (Study Group)	Check to make sure that the user has permission to send the message in that study group. Treat the message as string only and check to make sure it has no errors in processing, encrypt the message and send to the server, then decrypt the message and re-encrypt it separately for all users in that studygroup.	Show the message to all users in that StudyGroup.

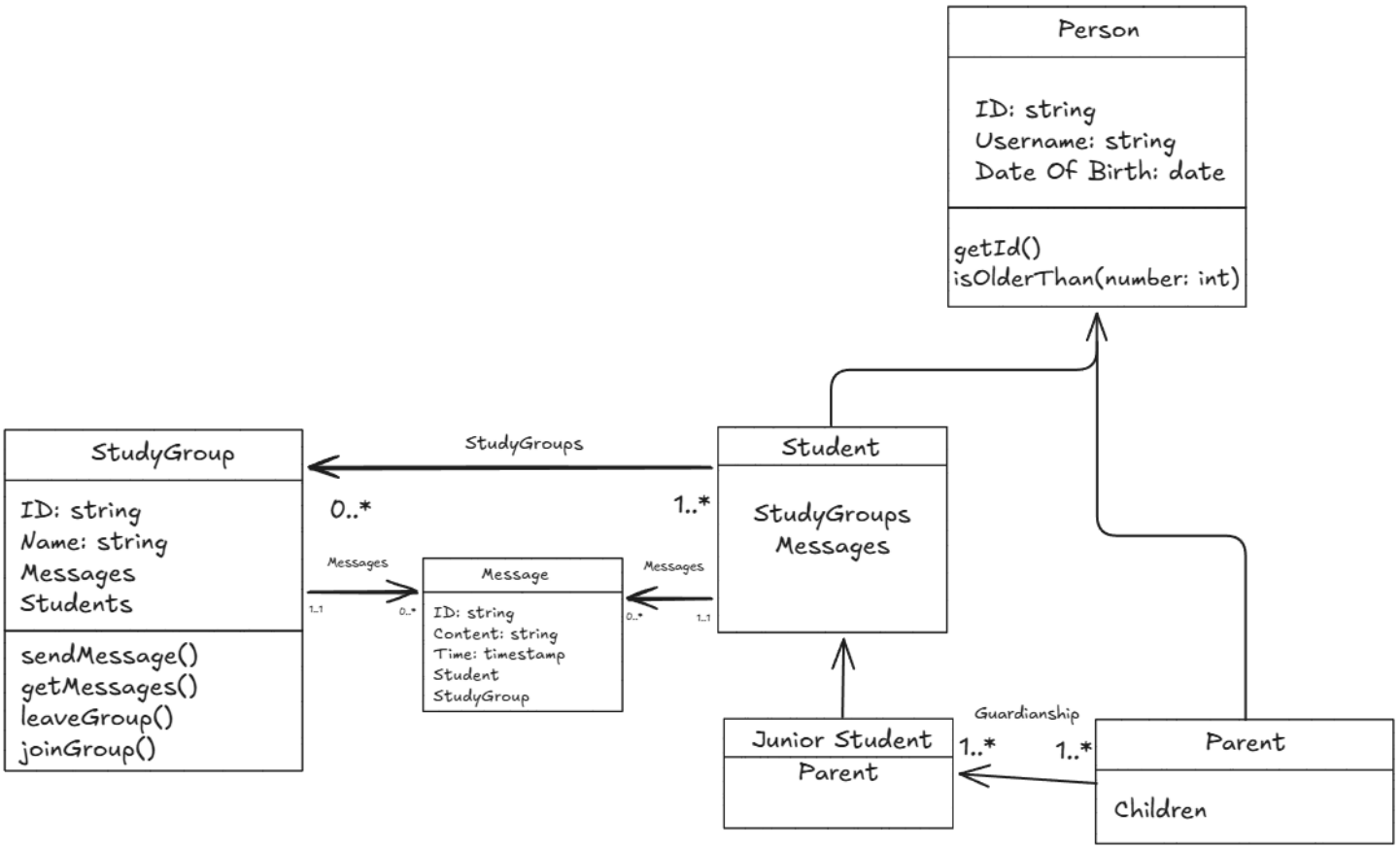
3.5 Data Dictionary

Variable	Data Type	Format for Display	Size in bytes	Size for display	Description	Example	Validation
Username	String	XX..XX	15	15	Username of the user	JamieRH20	Only alphanumerics and underscores
PasswordHash	String	XX..XX	32	N/A	Password hash of the user (password will never be sent across the server)	e7cf3ef4f17c3999a94f2c6f612e8a888e5b1026878e4e19398b23bd38ec221a	A valid SHA256 hash.
StudentID	String	XX..XX	15	N/A	Student ID	eafjioaefoaOHE	Only alphanumerics and underscores,

							completely unique.
StudyGroup Code	String	XXXXNN	6	6	The Code needed to join a StudyGroup	AbEf23	Only alphanumerics, completely unique.
StudyGroup ID	String	XX..XX	15	N/A	StudyGroup ID (for reference)	ffOIheagoi802	Only alphanumerics and underscores, completely unique.
StudyGroups	Array (String)		15 * Number of StudyGroups	N/A	List of StudyGroups that user can join	ffOIheagoi802 FEAo2b3910	Valid StudyGroupID
Message	String	XX..XX	750	750	Message content as a string	Yeah, so for that question make sure to deriverate pi.	Can be decoded by UTF-8

3.6 UML Class Diagram (if OOP)

Include a class diagram if your project uses an OOP approach.



Explain the class structure and relationships.

In this class diagram there are several key classes, the main class is the Person class which is the parent of both the “Parent” class and the “Student” class (As well as the “Junior Student” class by going up the chain). The person class keeps the data for any user in the system, holding the ID of the user, their username, and date of birth, something every single user will require. Then the chain splits into Students and Parents, Every student has from 0 to an infinite number of StudyGroups and Messages. Studygroups contain a list of students and messages and have methods for sending, receiving, leaving, joining etc… Messages just contain details about themselves and require a student and study group to exist. Junior students (Students under 16 years old) introduce parents, Junior students will need a parent to be in every group that they are in, and every junior student needs at least one parent, and every parent needs at least one junior student. All of this together leads to a system where we can have students of varying permission levels and parents with moderation permissions access StudyGroups and messages within the system.

---

4. Producing and Implementing

4.1 Development Process

Describe how the solution was built and implemented.
Justify the engineering techniques used, such as:

- Modular design

- Object-oriented principles
- Reuse of code
- Validation and error handling

StudyCircles was built and implemented through sprints and breaking the project down into two different sections, Functional and Graphical. I used modular design techniques alongside object-oriented principles within the functional half of the program so it could easier be implemented into the graphical side. Test methods were used -- and deleted for the final version of the program -- to validate all functions and ensure minimal errors. Error handling was also implemented for predictable errors such as user input mistakes (i.e. trying to sign in when they really should be registering etc…)

4.2 Key Features Developed

Describe the core features of the system.
Justify their inclusion.

Study Circles has various core features, all of which revolve around its network handling via sockets, This allows the program to talk from the client to server, this mixed with the encryption handling allows for StudyCircles to remain a secure & safe alternative for school students from social media as outlined in the original plan. The other core features -- the main ones seen by the user -- is messaging and invites, messages allow users to communicate with one another over an encrypted channel, and invites allow users to find each other for said communication.

4.2.1 Back-End Engineering Contribution

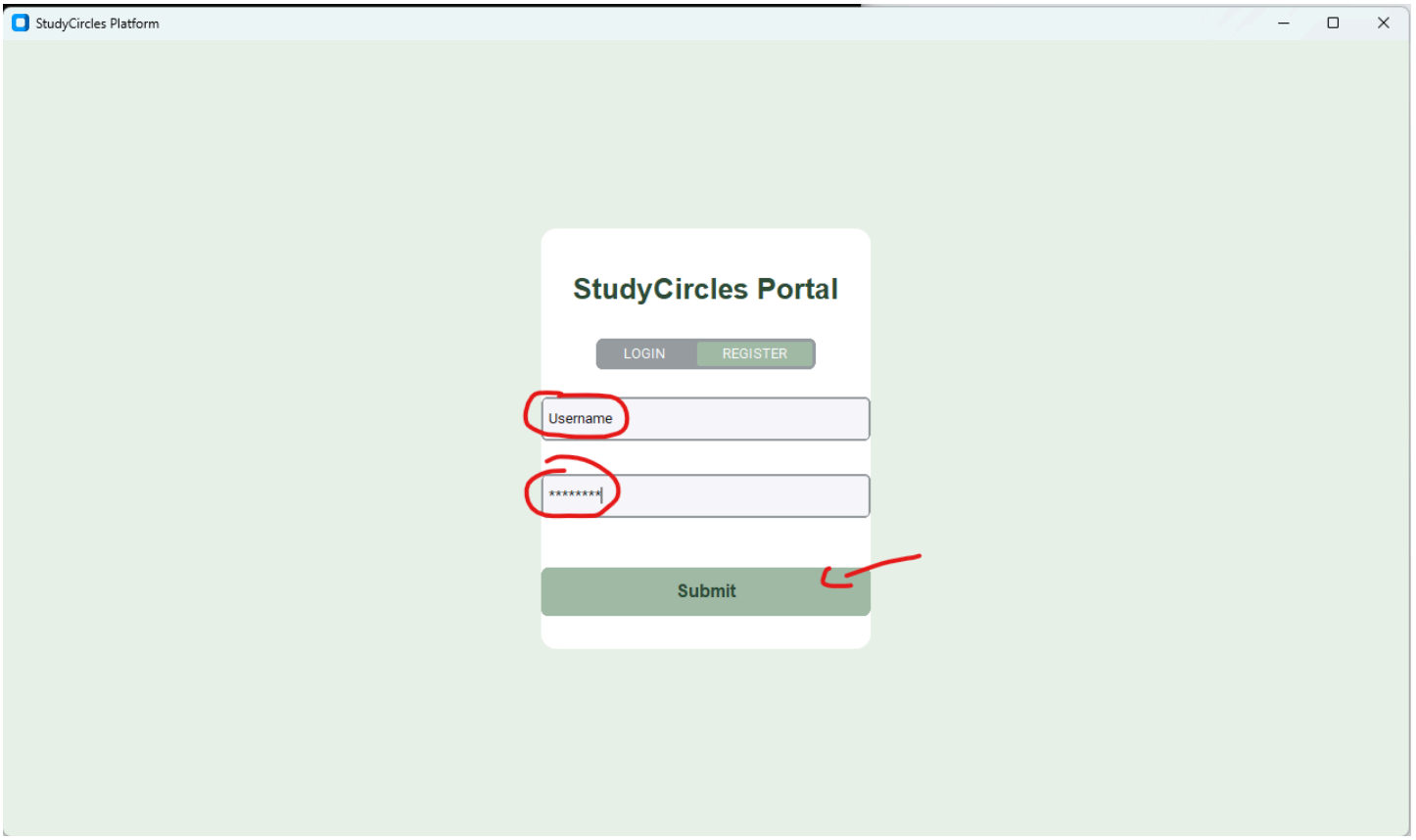
Explain how back-end engineering contributed to the success and ease of use of the software, including
- Data processing
- Validation and logic
- Storage and retrieval
- Authentication (if applicable)

Back-end engineering contributed to the success and ease of use of the software to an immense extent. Being a network-based application: Study Circles required extensive back-end engineering and forethought to allow for effective communication, from how packets should be sent, and how those received packets should be handled categorised -- a.k.a Data processing. This simple flow-based system in the back-end allowed for validation and logic to remain simple and easy to understand on a programming level, keeping the program predictable for users enhancing its ease of use. The modular and Object-Oriented approach of the programs Back-End engineering also helped with the retrieval and storage of data, allowing for past messages to be loaded just like new messages -- reusing the code and keeping the program efficient.

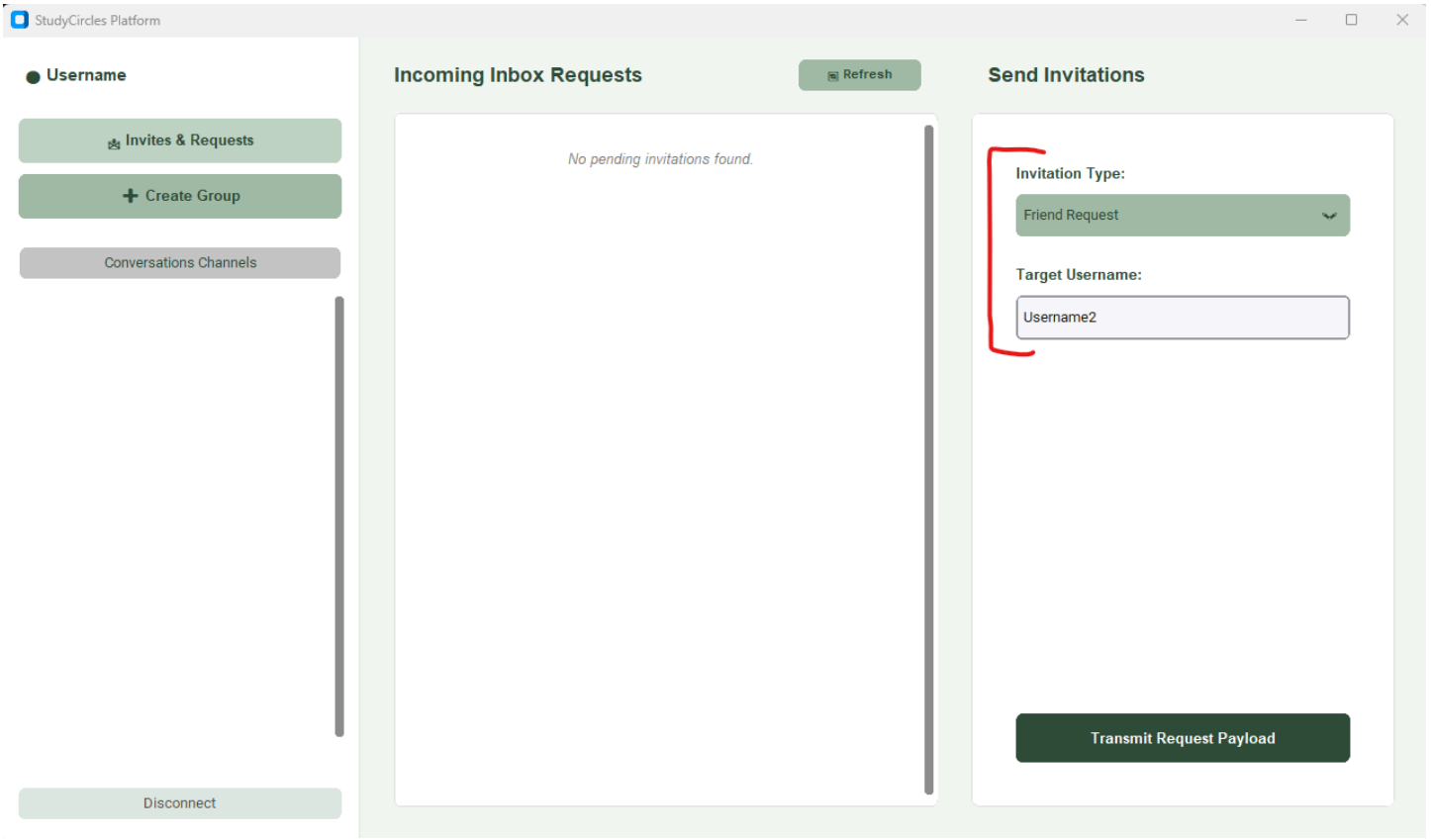
4.3 Screenshots of Interface

Include annotated screenshots explaining how the user interacts with the system.

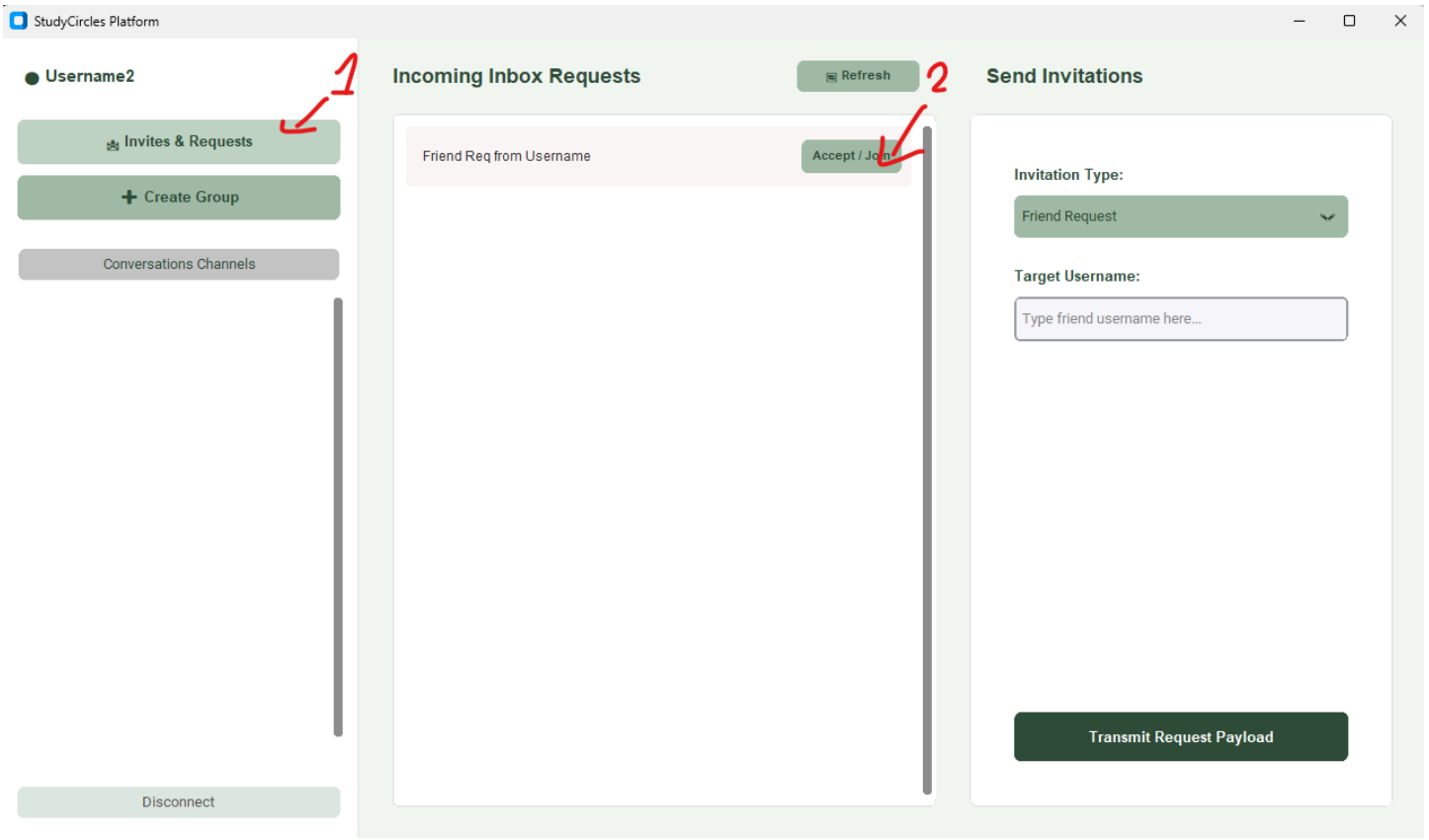
4.3.1 Login / Register



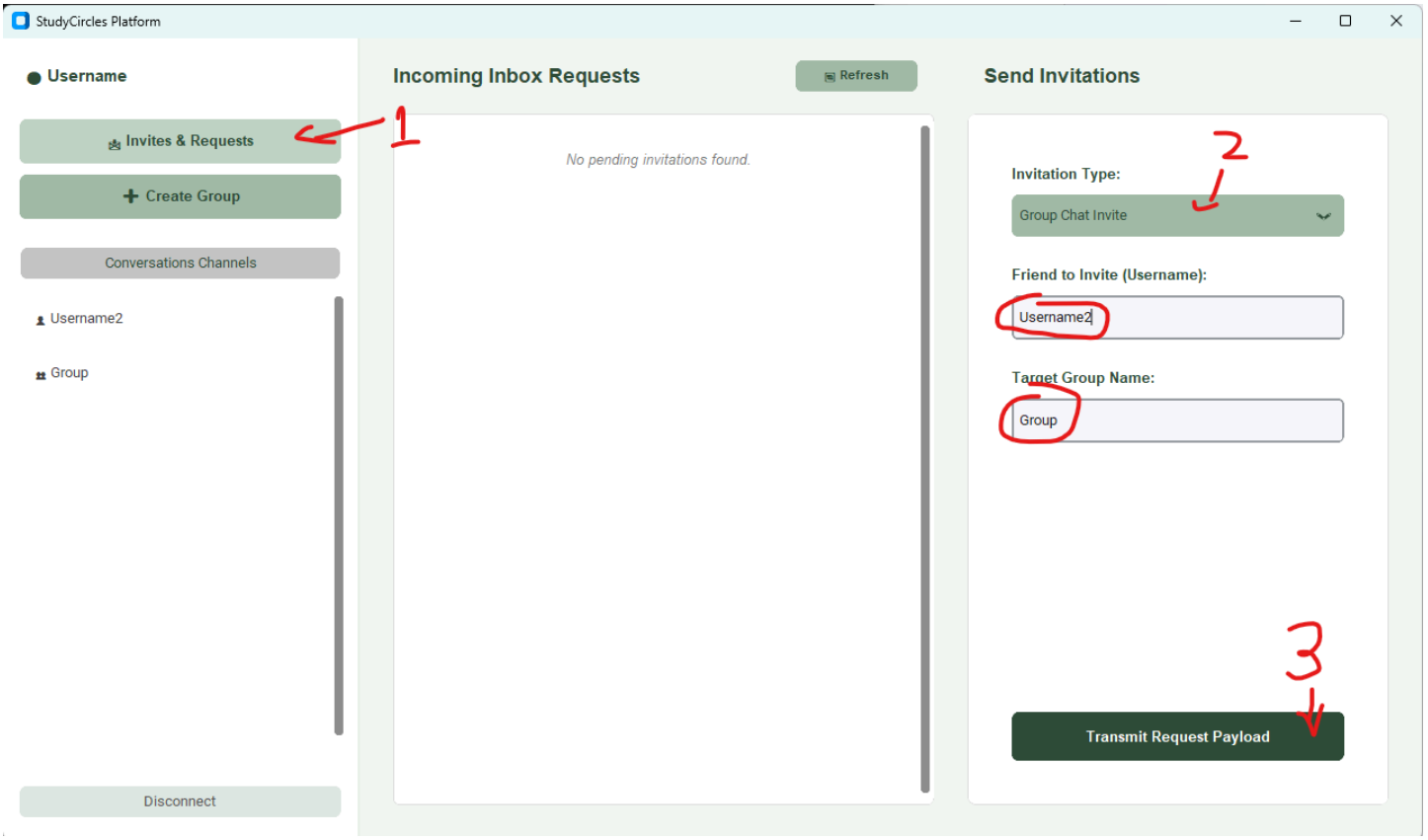
4.3.2 Friend Request



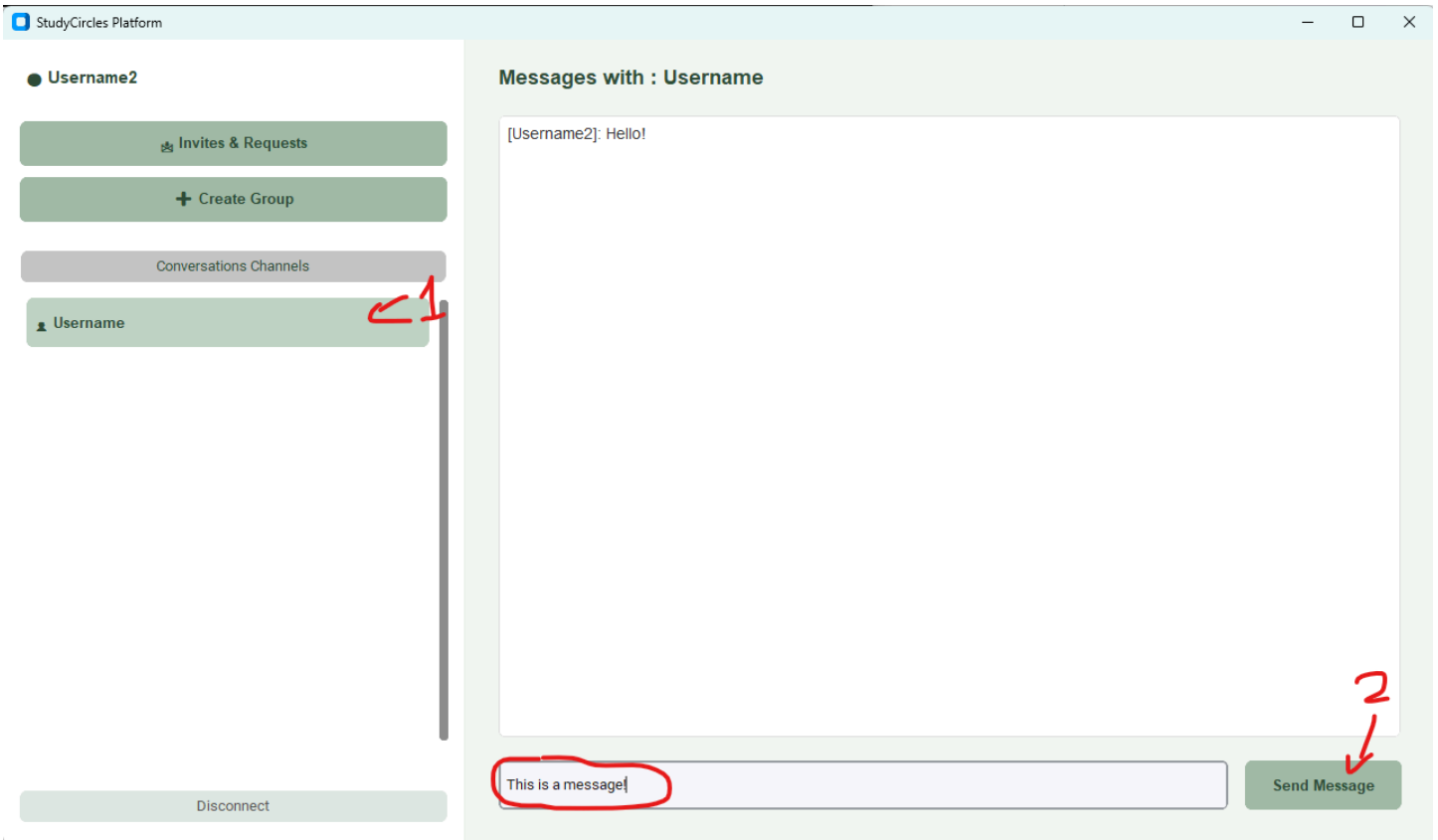
4.3.3 Accept Friend Request/Group Invite



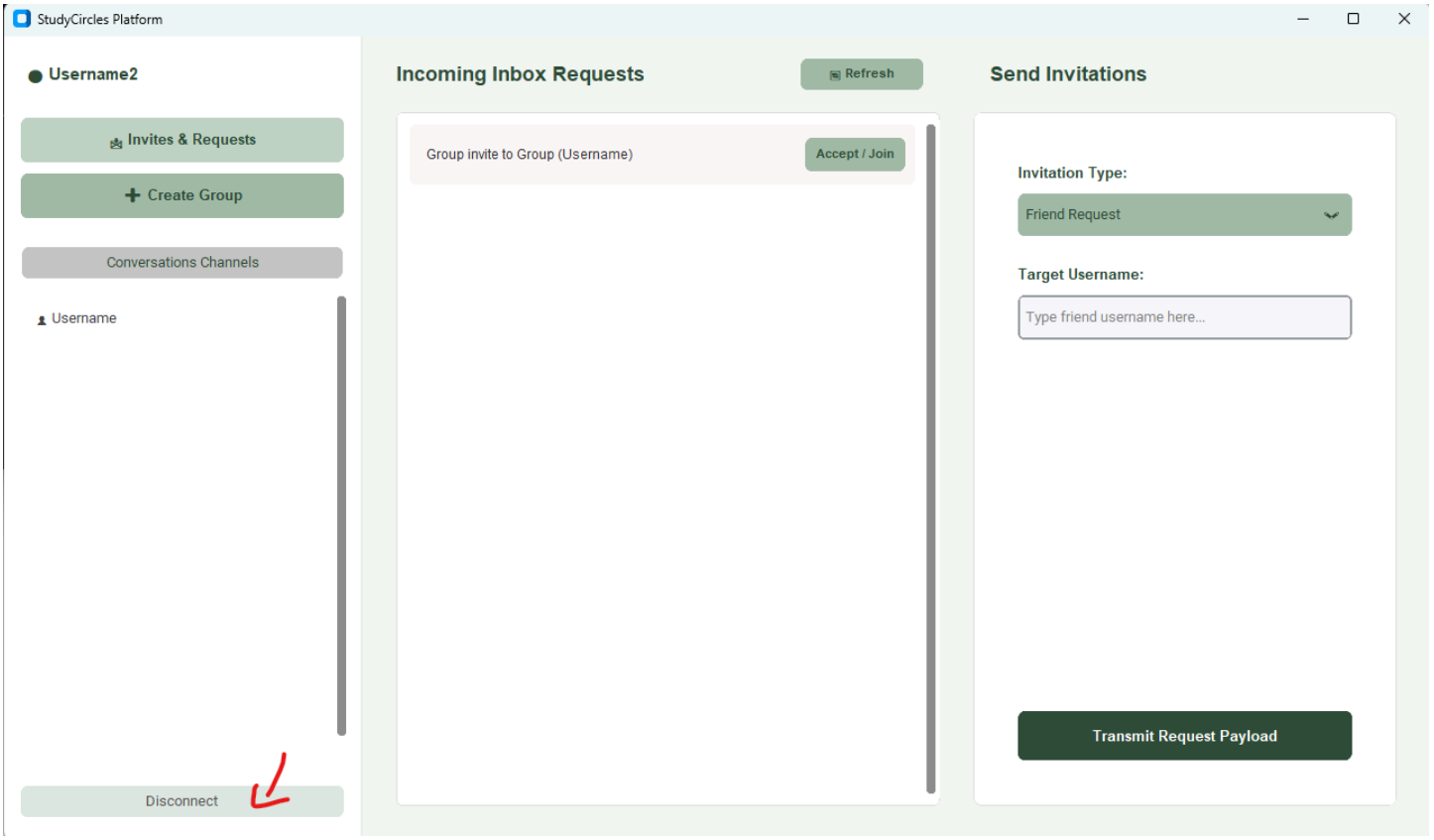
4.3.4 Group Invite



4.3.5 Send Group/Friend Message



4.3.6 Disconnect



4.4 Version Control Summary (Optional)

Summarise commits, iterations, or sprints if version control was used.

StudyCircles version control aligns very closely with the Gantt chart (provided in 2.3) however Login and Register was kind of merged into packets, with some more packet handling in between the two. 7 commits in total were made (6 if you disregard the initial commit). The first commit was part of the 3rd and 4th sprint -- Packet Handling and Logging In / Registering -- and laid out the backend for:

- Sending and Receiving Messages,
- Sending and Receiving Friend Requests and Group Invites
- Creating Groups
- Logging in and Registering
- File Messages

The next commit got rid of the client.py scripts that were for the TUI as it was getting hard to handle with that stage of development and removed all work on File Messages.
After that the 5th Sprint Started which got the:

- Login And Register Menu working

And started work on the:

- Group Invitations/Friend Request UI
- Chat Menu

Then there were a few basic commits that contributed to the previous code base.
The last commit focused on improving language -- as I had made everything technical language for ease of research after breaks between development.

5. Testing and Evaluation

5.1 Testing Methods Used

Describe testing approaches, such as:

- Unit testing
- Integration testing
- User testing

Explain how testing results were used to improve performance, efficiency, or reliability.

Unit Testing was used throughout development for each added function -- the test then being removed after validation, to ensure that the functions logic and code were working as intended. Integration testing was minimal as the systems themselves were quite simple. User testing was a great part of the programs development, every time a feature was added it was immediately tested by a user (Usually myself due to a lack of time to survey peers, but sometimes my family members and peers.), This was not just during the GUI sprint, but throughout the backend -- hence the inclusion of a TUI when interfacing directly with client.py instead of client_ui.py

5.2 Test Cases and Results

5.2.1 Test Cases

Test ID	Description	Expected Result	Actual Result	Pass/Fail
LM01	Valid login	Success message	Success message	Pass
LM02	Invalid login	Error message	Error message	Pass
LM03	Unique Registration	Success message	Success message	Pass
LM04	Already Registered Details	Error Message	Error Message	Pass
IM01	Valid User	Success message	Success message	Pass
IM02	Valid Invite	Success message / Invite Received	Success message / Invite Received	Pass
IM03	Valid Group	Success message	Success message	Pass
IM04	Invalid Group	Error message	Error message	Pass
CC01	Send Message	Message from user in chatbox	Message from user in chatbox	Pass
CC02	Receive Message	Message from user in chatbox	Message from user in chatbox	Pass
CC03	Load Saved Messages	Saved messages in chatbox	Saved messages in chatbox	Pass
CC04	Send group message	Message from user in group chatbox	Message from user in group chatbox	Pass

CC05	Receive group message	Messages from all users in group chatbox	Messages from all users in group chatbox	Pass
CC06	Receive Message while offline	Message shows up on log in	Message shows up on log in	Pass
GI01	Disconnect	Close and shut network connection	Closed and shut network connection	Pass
GI02	Create Group	Group gets added to channel list	Group gets added to channel list	Pass

5.2.2 Key

LM	Login Menu
IM	Invitations Menu
CC	Chat Channels
GI	General Interfacing

5.3 Evaluation Against Requirements

Evaluate how effectively the solution meets the identified functional and non-functional requirements.

The solution meets the identified functional and non-functional requirements to a high extent, however it does fall short in some places. While the solution does provide:

- Easily adding friends
- Sending Messages
- Receiving Group messages
- Quick and Near instant messaging
- Clean and simple visuals
- Encryption for all forms of messaging
- 100% Message success rate (Testing for this however was not extensive)

The solution does not:

- Use friend codes -- However for this case I think keeping everything to simple usernames works well and stops impersonation which could be dire in a student-focused study-messaging app.
- Chat moderation
- Blocking users
- Uploading files
- Age Selection -- However again this could be a good thing to stop for age sharing
- Basic Sounds -- As it did not add to user experience enough to be worth the extra multi-threading effort.

5.4 Improvements and Future Development

Outline your project's limitations.
Explain realistic future enhancements.

The project has various limitations which led it to have diminishing successes in comparison to its ambitious original goals. These include; Time given for the project amongst other assessment tasks, ROI (Return on investment) on programming heavy implementations -- such as adding sounds, friend codes, and uploading files, and school based network restrictions -- which stopped the project from ever being capable of bridging across multiple networks.
Realistic future enhancements would need to include the following:
Sound design for more intuitive and instant feedback,
File support for furthering the study capable of the site (However at the moment users *can* send links to downloads)
Blocking and Chat moderating (This would be VITAL to making the program truly secure for other users, however this is also a big time sink and eats into some of what makes the program secure -- which is partly why other encrypted messaging programs such as WhatsApp avoid these features).

---

6. Feedback, Security and Reflection

6.1 Summary of Client or Peer Feedback

Summarise feedback received and explain how it influenced development.
You could collect a 'PMI' (Plus, Minus, Implication) table from at least three different people after testing, or record and summarise an interview with at least three three people who test the software.

Over the course of development I collected various forms of feedback from my peers and family members, primarily just short thoughts about what I could improve, and used this to better develop StudyCircles. From small things such as colour and language choices but also some bigger changes like restricting group invites to only those you are friends with. Feedback is vital to development in all of its stages, from design to polishing, this can be seen in other popular projects -- namely the original subnautica -- and I hope this is also reflected in the final product of StudyCircles.

6.2 Secure Software Design and Data Handling

Evaluate the approach undertaken to safely and securely collect, use, and store data.
Your evaluation should address:

- Secure coding practices applied during development
- Input validation and error handling
- Data storage and protection methods
- The impact of secure software design on user trust, data integrity, and system reliability

StudyCircles prides itself on its approach to safely and securely collecting, using and storing data, having all packets be encrypted across a private key, with the server only saving necessary data. An example of this practice is how StudyCircles handles offline messages, with the server storing a copy of the encrypted message, and then relaying it once the user logs back on, deleting the server copy immediately after.
StudyCircles also focuses on Input Validation and error handling, providing a clean user experience through

alerts in the UX to inform the user when they've made a mistake -- this is namely in the Invite and Log In System. All key data stored on the server is protected as well, with the Log In system having passwords hashed on the client side -- so the server only ever sees that hash -- keeping their password secure. These secure software design techniques impact the amount of trust users can place in StudyCircles, keeping that trust with its Data Integrity and System reliability throughout the program's modular design. In short StudyCircles has an exemplary approach to safely and securely collecting, using and storing data as a program; prioritising the impacts these Secure Software Design choices will have on user trust, data integrity and system reliability.

6.3 Personal Reflection

Reflect on what you learned during the project, including

- Software engineering skills developed
- Challenges encountered and how they were overcome

Over the course of this project I've developed many skills, As stated previously in this document I've had to learn various libraries -- Mainly the sockets library. When it comes to networking, system architecture is extremely important - even the slightest offset in wording and design choices can snowball into huge issues. As such, StudyCircles helped me develop my skills in modular programming, working with classes in python, and keeping a consistent design language.
StudyCircles' development was not without challenges; however, the original architecture for packets took me about 4 tries to get working properly. Scope Creep was also a common issue throughout StudyCircles' development -- namely with File sending (which was far too complicated for me to pull off) and the level to which security would be maintained (Handshakes, and proper server by-passed encryption were quite difficult and often unreliable so they were minimised).
Overall StudyCircles' was a major step in furthering my skills in developing projects throughout every stage; If I was to develop StudyCircles' again, I wouldn't, It's too large of a project for a solo developer to pull off reliably, and the complex systems it requires to function lend themselves to lower level languages that work faster and more consistently than python -- That is not to say that StudyCircles' development was not a net positive experience though.